

Rapport de Mini-Projet

Simulation d'un Ascenseur Multithreading

Membres du binôme :
ELASRI Saad (Responsable Logique & Multithreading)
NAMRANI Ilyas (Responsable Architecture & Interface Swing)

Date de remise : 22 mai 2026

Module : Programmation Concurrente / Java

1 Introduction

Ce mini-projet a pour objectif de concevoir et développer la simulation d'un ascenseur desservant un immeuble de 5 étages (du rez-de-chaussée au 4ème étage). L'enjeu principal réside dans la gestion de la concurrence (multithreading) : plusieurs entités (personnes, ascenseur, interface graphique) agissent simultanément et doivent partager des ressources sans créer de conflits ou d'interblocages.

2 Architecture Globale

Le projet a été pensé selon une approche Orientée Objet. L'application est divisée en plusieurs entités distinctes qui interagissent entre elles :

- **Modèle** : `Personne`, `Etage`, `Immeuble` (Gestion des données et des files d'attente).
- **Contrôleur/Moteur** : `Ascenseur` (Thread principal), `GenerateurPersonnes` (Thread secondaire).
- **Vue** : `SimulationGUI` (Interface Java Swing affichant l'état en temps réel).

3 Répartition des Tâches

Pour optimiser notre temps et éviter les conflits lors du développement, nous avons divisé le projet en deux grands pôles distincts et équilibrés :

3.1 Lot A : Moteur, Synchronisation et Logique Métier (Réalisé par Saad)

Ce lot se concentre sur le cœur de la simulation et la gestion des threads.

- Sécurisation des ressources partagées (files d'attente des étages) via les moniteurs (`synchronized`, `wait`, `notifyAll`).
- Gestion de la capacité maximale de l'ascenseur (4 personnes) à l'aide d'un `Semaphore`.

- Implémentation de l'algorithme de déplacement intelligent de l'ascenseur (continuité de direction, changement de sens aux extrémités).

3.2 Lot B : Architecture Objet, UI et Génération (Réalisé par Ilyas)

Ce lot se concentre sur l'interaction utilisateur et la dynamique de la simulation.

- Conception des classes de base (*Personne*, *Direction*) et du thread de génération aléatoire des passagers.
- Développement de l'interface graphique avec **Java Swing** (dessin de l'immeuble, de la cabine, des files d'attente et des indicateurs de direction).
- Gestion de la boucle de rafraîchissement (animation) pour lier les données du modèle (Lot A) à l'affichage visuel sans bloquer l'Event Dispatch Thread (EDT) de Swing.

4 Détails Techniques et Choix d'Implémentation

4.1 La Synchronisation des files d'attente (Moniteurs)

Les étages possèdent chacun une file d'attente (*Queue<Personne>*). Cette liste est modifiée à la fois par le Générateur (qui ajoute des personnes) et par l'Ascenseur (qui les récupère). Pour éviter les accès concurrents (Race Conditions), les méthodes *ajouterPersonne()* et *recupererPassager()* utilisent le mot-clé *synchronized*. Cela garantit qu'un seul thread peut modifier la file d'un étage à un instant donné. L'utilisation de *notifyAll()* permet de réveiller l'ascenseur s'il était en attente d'appels.

4.2 Le respect de la capacité maximale (Sémaphores)

L'énoncé impose une limite stricte de 4 personnes dans la cabine. Nous avons utilisé un *Semaphore(4, true)* (équitable) dans la classe *Ascenseur*.

- **Embarquement** : L'ascenseur tente d'acquérir un permis avec *tryAcquire()*. S'il réussit, la personne monte. Sinon, l'embarquement s'arrête.
- **Débarquement** : Lorsqu'une personne arrive à destination, elle est retirée de la liste et le permis est rendu via *release()*.

4.3 Stratégie de déplacement de l'ascenseur

L'ascenseur tourne dans une boucle infinie (*while(true)*). Son comportement est régi par les règles suivantes :

1. Il conserve sa direction (MONTEE ou DESCENTE) tant qu'il a des passagers allant dans ce sens.
2. Il inverse sa direction lorsqu'il atteint les étages extrêmes (0 ou 4).
3. S'il est vide, il se met à l'ARRÊT jusqu'à ce qu'un nouvel appel soit détecté sur un étage.

4.4 L'Interface Graphique (Swing)

L'interface a été conçue pour être claire et réactive. Le thread de l'interface graphique (EDT) se met à jour périodiquement en interrogeant l'état des objets (étage courant de l'ascenseur, taille des files d'attente, etc.). Une attention particulière a été portée pour que les calculs de l'ascenseur (*Thread.sleep*) ne gèlent pas l'interface visuelle.

5 Défis Rencontrés et Solutions

- **Problème** : Interblocage (Deadlock) lors de l'embarquement si l'ascenseur attendait indéfiniment des personnes alors que le générateur était en pause.
- **Solution** : Utilisation de `tryAcquire()` sur le sémaphore plutôt que `acquire()`, et vérification des conditions avant la mise en attente.
- **Problème** : L'interface graphique figeait lors du déplacement de l'ascenseur.
- **Solution** : Séparation stricte entre le thread de l'ascenseur (qui fait des `sleep()` pour simuler le mouvement) et le thread de Swing (`SwingUtilities.invokeLater / Timer`).

6 Conclusion

Ce projet nous a permis de mettre en pratique les concepts théoriques de la programmation concurrente en Java. La séparation des tâches entre la logique métier (Lot A) et l'interface/génération (Lot B) s'est avérée être une excellente stratégie pour avancer efficacement. L'ascenseur fonctionne désormais de manière fluide, sans erreur de concurrence, et l'interface permet de bien visualiser le comportement du système.